

Aula 4 — Maestro Mobile Cross-Platform

"Appium ou Maestro?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 15/10/2026

Recap da Aula 3

- SPA hydration testing
- Service Worker tests (cache, lifecycle)
- PWA offline + manifest validation
- Lighthouse CI com performance budgets

Lab PWA Testing entregue?

Agenda da aula (3h30)

- **90min** — Maestro mobile: YAML declarativo, comparativo vs Appium/Detox, cloud farms (teoria + demo guiada)
- **15min** — Intervalo
- **90min** — Lab hands-on em sala: suíte Maestro (live coding com replicação pelos alunos)
- **15min** — Wrap-up

Objetivos M4

- **Implementar** suítes Maestro com YAML declarativo
- **Aplicar** conditional flows e fragmentos reutilizáveis
- **Comparar** Maestro vs Appium vs Detox em matriz quantitativa
- **Executar** flows em Firebase Test Lab
- **Mitigar** flakiness mobile específico (animações, popups, biometria)

Por que Maestro venceu

Reddit, Cash App, Headspace, Loop adotaram em 2023+.

Razões técnicas:

1. **YAML declarativo** vs código imperativo
2. **Retry built-in** elimina sleeps
3. **Sem WebDriver** — fala direto com device
4. **Setup minutos**, não dias
5. **Cross-platform real** — mesmo flow iOS + Android

Maestro — primeiro flow

```
appId: com.myapp
---
- launchApp
- tapOn: "Email"
- inputText: "user@email.com"
- tapOn: "Senha"
- inputText: "secret"
- tapOn: "Entrar"
- assertVisible: "Bem-vindo"
```

```
maestro test login_flow.yaml
```

30 segundos de instalação, segundo teste E2E rodando.

Maestro — comandos principais

```
- launchApp
- tapOn: "texto"
- tapOn: { id: "submit-button" }
- inputText: "hello"
- swipe: { start: "50%, 80%", end: "50%, 20%" }
- scroll
- assertVisible: "Welcome"
- assertNotVisible: "Loading"
- pressKey: "BACK"
- waitForAnimationToEnd
- takeScreenshot: "after-login"
```

Conditional flows

```
appId: com.myapp
---
- launchApp
- runFlow:
  when:
    visible: "Aceitar cookies"
    file: ./flows/dismiss-cookies.yaml
- runFlow:
  when:
    notVisible: "Welcome"
    file: ./flows/login.yaml
- assertVisible: "Welcome"
```

Suíte se adapta ao estado do app sem `if` imperativo.

runFlow — fragmentos reutilizáveis

```
# flows/login.yaml
- tapOn: "Email"
- inputText: "user@email.com"
- inputText: "secret"
- tapOn: "Entrar"
```

```
# tests/checkout.yaml
appId: com.myapp
---
- launchApp
- runFlow: ../flows/login.yaml
- tapOn: "Carrinho"
- tapOn: "Comprar"
- assertVisible: "Pedido confirmado"
```

DRY em testes E2E.

Parametrização via env

```
appId: com.myapp
env:
  EMAIL: "default@email.com"
---
- launchApp
- inputText: ${EMAIL}
```

```
maestro test login.yaml -e EMAIL=admin@myapp.com
```

Maestro Studio

```
maestro studio
```

Interface gráfica:

- Hierarquia do app em tempo real
- Tap visual gera comando YAML
- Replay com inspeção

| Excelente onboarding pra iniciantes.

Comparativo Maestro vs Appium

Critério	Maestro	Appium 2
Linhas (mesmo flow)	~10	~30+
Tempo execução	~3s	~7–12s
Flake rate inicial	< 5%	~15–25%
Setup	10min	1–2h
Curva	Plana	Íngreme
Apps nativos	✓	✓
Apps RN	✓	⚠ flake
Drivers especializados	✗	✓
Selenium Grid integration	✗	✓

Maestro é melhor padrão. Appium ainda relevante em casos específicos.

Detox (RN específico)

Gray-box. Roda no mesmo processo do app RN.

```
describe('Login', () => {  
  beforeEach(async () => {  
    await device.reloadReactNative();  
  });  
  
  it('navigates to Home after valid login', async () => {  
    await element(by.id('email')).typeText('user@email.com');  
    await element(by.id('password')).typeText('secret');  
    await element(by.id('submit')).tap();  
    await waitFor(element(by.id('homeScreen'))).toBeVisible().withTimeout(5000);  
  });  
});
```

Para RN exclusivo, Detox compete com Maestro. Para nativo, Detox não cobre.

Cloud Devices — opções 2026

Plataforma	Free	Forte em
Firebase Test Lab	100min/mês virtual + 5h físico	Apps Android, integração Google
BrowserStack App Live	30min trial	Manual + alto volume
BrowserStack App Automate	Pago	Automação cross-platform
LambdaTest	30min trial	Alternativa BrowserStack
Sauce Labs	Pago	Enterprise
AWS Device Farm	1000min device-min free	Ecossistema AWS

Firebase Test Lab — exemplo

```
gcloud auth login
gcloud config set project meu-projeto

# Listar models do catálogo Test Lab antes de submeter
gcloud firebase test android models list

# Submeter (use model IDs do catálogo, não nomes AVD)
gcloud firebase test android run \
  --type instrumentation \
  --app app/build/outputs/apk/debug/app-debug.apk \
  --test app/build/outputs/apk/androidTest/debug/app-debug-androidTest.apk \
  --device model=shiba,version=34,locale=pt_BR \
  --device model=redfin,version=30,locale=pt_BR \
  --timeout 5m
```

Reports em <https://console.firebase.google.com/>. Models reais: shiba (Pixel 8), redfin (Pixel 5), etc.

Estratégia top devices (StatCounter)

Cobrir top 10–15 devices = ~80% da base:

```
Brasil 2026:  
1. Galaxy A15 — 8%  
2. Moto G54 — 6%  
3. iPhone 13 — 5%  
4. Galaxy A55 — 5%  
5. iPhone 14 — 4%  
6. Redmi Note 13 — 4%  
... etc
```

Variar versões OS (minSdk + atual + beta) é tão importante quanto modelos.

Flakiness mobile específica

Causa	Mitigação
Animações	<code>waitForAnimationToEnd</code>
OS popups (permissões)	Granting via flow
Biometria	Mock em test config
Network jitter	Mock server
Lifecycle (bg/fg)	Estabilizar antes de assert
Touch latency emulator	Real device confirmação

IA aplicada — geração Maestro via Claude

```
const userStory = "Como user, quero comprar produto e ver toast de confirmação.";

const response = await anthropic.messages.create({
  model: 'claude-opus-4-7',
  max_tokens: 1024,
  messages: [{
    role: 'user',
    content: `Gere flow Maestro YAML para esta user story:\n${userStory}\n\nUse fragmentos quando apropriado. Inclua conditional flow se necessário.`
  }],
});
```

Bom em **transformação** (story → flow). Sempre revisar antes de aceitar.

Lab — suíte Maestro completa (90min)

Roteiro

1. Setup Maestro CLI (5min)
2. Primeiro flow funcionando (10min)
3. 5 flows com fragmentos + conditional (25min)
4. Mesmo flow em Appium para comparar (15min)
5. Submeter para Firebase Test Lab (15min)
6. Analisar relatório + flake rate (10min)
7. Discussão (10min)

Pitfalls comuns

- ✗ `tapOn:` por texto que muda — flake em A/B test ou tradução
- ✗ Não usar `runFlow` para login — repetição massiva
- ✗ Suíte sem retry — flake transitório vira falha
- ✗ Só emulator — esconde issues touch latency
- ✗ Não monitorar flake rate — degrada silenciosamente

Atividade — Lab Maestro Mobile (15 pts)

Entrega: ver Canvas.

Entregar repo com:

1. **5 flows funcionais em iOS** (Simulator) — 4pts
2. **5 flows funcionais em Android** (Emulator) — 4pts
3. **Flakiness < 5%** em 3 runs consecutivos — 3pts
4. **Matriz comparativa Maestro vs Appium** (linhas, tempo, sucesso, 2pgs) — 4pts

| Critério eliminatório: ao menos 1 flow em device físico (Firebase Test Lab).

Leituras obrigatórias

- **Maestro Docs** (oficial)
- **Linares-Vásquez et al. (2017)** — *Continuous, Evolutionary and Large-Scale*. ICSME
- **Kochhar et al. (2015)** — *Understanding the Test Automation Culture*. ICST
- **Mobile.dev Blog** — *Why YAML for E2E Tests*

Próximo módulo (M5 — FLAGSHIP)

IA Avançada em Testes

- Visual AI (Applitools)
- Geração de testes via Claude API
- Self-healing (Healenium)
- Agentes (Browser Use, Computer Use)

Módulo flagship. Onde IA reescreve QA.

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith